TECHNICAL CASE STUDY REPORT • CLOUD INFRASTRUCTURE & SYSTEMS ENGINEERING

Exploration of Cloud Service Models (IaaS, PaaS, SaaS) using a Free-Tier Cloud Platform

Target Platform: AWS / GCP Free Tier **Author:** Cloud Systems Architect **Document Ref:** CS-CLM-2026-V1

1. EXECUTIVE AIM & RESEARCH OBJECTIVES

Primary Aim

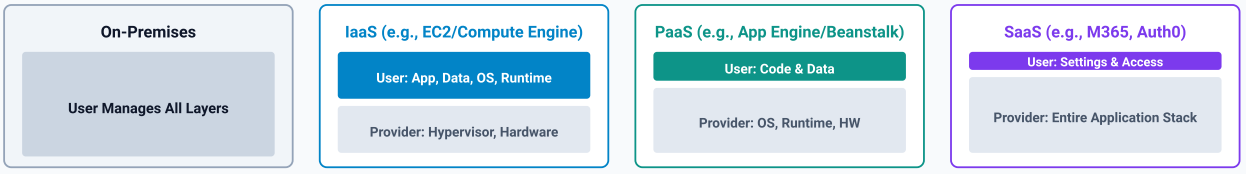
To practically explore, deploy, and evaluate the three primary cloud service delivery models—Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS)—utilizing a zero-cost cloud free-tier account to assess resource control, governance, and operational trade-offs.

Core Objectives

- Analyze shared responsibility boundaries across cloud paradigms.
- Provision virtual infrastructure (IaaS) and runtime applications (PaaS).
- Integrate managed cloud utility services (SaaS).
- Evaluate cost limits, scalability, and maintenance burdens.

2. CONCEPTUAL FOUNDATIONS & SHARED RESPONSIBILITY

Cloud computing abstracts physical hardware into flexible, service-based resources. Selecting the correct service model depends on balancing administrative control against operational convenience. Under the *Shared Responsibility Model*, management duties shift progressively from the consumer to the cloud provider as you move from IaaS to SaaS.



3. FUNDAMENTAL OVERVIEW OF CLOUD SERVICE MODELS

IaaS Tier

Infrastructure as a Service

Provides raw virtualized computing resources (VMs, block storage, virtual networks). Users configure operating systems, patches, network routing, and security groups.

Free Tier Limits: 750 hrs/mo micro instance, 30GB EBS storage.

PaaS Tier

Platform as a Service

Delivers a managed execution environment. Developers push application code while the cloud platform handles autoscaling, web servers, and OS maintenance.

Free Tier Limits: 28 instance-hours/day, free SSL domain routing.

SaaS Tier

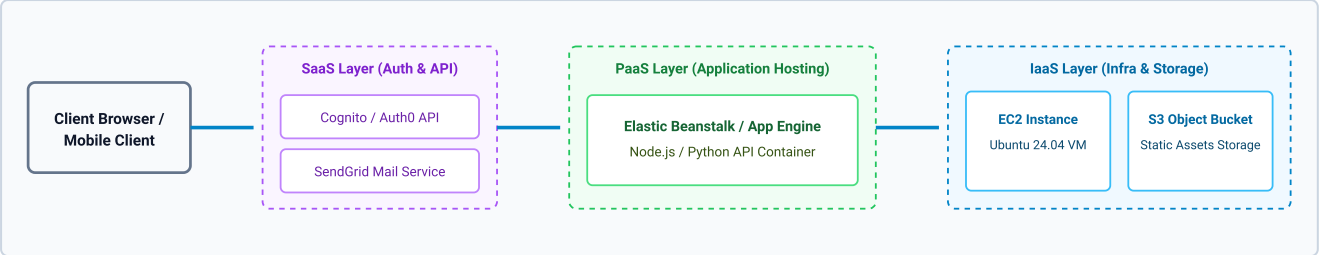
Software as a Service

Offers ready-to-use software applications accessible via web browsers or APIs. The vendor manages hardware, database engines, updates, and scalability end-to-end.

Free Tier Limits: Up to 50k monthly active users / API call quotas.

4. PRACTICAL DEPLOYMENT TOPOLOGY ON FREE-TIER CLOUD

To understand cloud service integration, a multi-tier prototype architecture was constructed using standard Cloud Free Tier allowances. The architecture leverages IaaS for storage/compute processing, PaaS for hosting microservice APIs, and SaaS for authentication and security auditing.



5. IMPLEMENTATION PLAN & CONFIGURATION LIFECYCLE

1. **IaaS Provisioning (Virtual Machine & Block Storage)**
   Launched an AWS EC2 `t2.micro` Linux instance within a custom Virtual Private Cloud (VPC). Configured Security Group rules to restrict SSH access to specific IP ranges and opened HTTP/HTTPS ports. Attached a 20GB EBS storage volume for persistence.

2. **PaaS Deployment (Managed Runtime Environment)**
   Deployed a Node.js REST API service using AWS Elastic Beanstalk / GCP App Engine. Source code was pushed directly from a GitHub repository. The platform automatically provisioned load balancers, configured domain mapping, and handled runtime scaling.

3. **SaaS Integration (Managed Authentication & Monitoring)**
   Integrated AWS Cognito for user directory management, token authorization, and OAuth2 registration. System logging and metric alarms were delegated to CloudWatch, eliminating the need to write custom telemetry pipelines.

> **Key Practical Observation: Free-Tier Governance**
> Operating under free-tier conditions requires strict governance to prevent accidental charges. Critical setup steps include configuring AWS Billing Alerts at $1.00 thresholds, selecting standard EBS storage over Provisioned IOPS, and terminating non-essential Elastic IP addresses when VM instances are halted.

6. COMPARATIVE TRADE-OFF EVALUATION MATRIX

The following table synthesizes empirical observations gathered during hands-on deployment across the three cloud service paradigms.

Evaluation Metric	IaaS (Infrastructure)	PaaS (Platform)	SaaS (Software)
Control Level	Maximum; OS kernel, networking & storage configuration.	Moderate; application settings & environment variables.	Minimal; restricted to application settings & user roles.
Deployment Speed	Slower; requires OS setup, updates & software stack installation.	Fast; direct deployment of application code/containers.	Instantaneous; immediate account provisioning & API access.
Maintenance Burden	High; OS patching, firewall maintenance & security fixes.	Low; platform vendor patches runtime & OS automatically.	Zero; full maintenance owned by software vendor.
Free Tier Viability	High (750 EC2 hours/mo suitable for single VM testing).	High (Generous daily free instance-hour quotas).	Variable (Free tiers often feature user/limit caps).
Vendor Lock-in	Low; VM images easily migrate across cloud providers.	Moderate; platform-specific deployment scripts/manifests.	High; proprietary data formats & ecosystem lock-in.
Primary Use Case	Custom servers, database hosting, legacy migration.	Rapid web app deployment, microservices, mobile APIs.	Email, CRM, identity management, team collaboration.

7. REAL-WORLD CASE APPLICATION: FOOD DELIVERY MICROSERVICES

To illustrate practical selection, consider a multi-tiered system like an **Online Food Delivery System** (e.g., Swiggy/Zomato archetype):

Hybrid Architectural Allocation

- **IaaS:** Runs high-performance geospatial routing algorithms & Redis cache clusters on dedicated VMs.
- **PaaS:** Hosts mobile gateway REST APIs and order fulfillment microservices with auto-scaling.
- **SaaS:** Integrates Stripe for payments, Twilio for SMS alerts, and Auth0 for secure user logins.

99.95%

Target Availability Achievable

Combining PaaS elasticity with SaaS resiliency maximizes uptime while minimizing engineering maintenance cost.

8. KEY FINDINGS & STRATEGIC CONCLUSION

The hands-on exploration confirms that no single cloud service model fits all operational needs. **IaaS** offers unmatched control and portability but demands continuous system administration. **PaaS** dramatically accelerates developer productivity by stripping away infrastructure management, making it ideal for Agile development sprints. **SaaS** delivers instant utility, freeing teams to focus on core business logic rather than rebuilding standard features like authentication or payment processing.

> **Final Recommendation:**